DQN-Based Competitive Tetris Agent

Kuang-Yi Tai
MSCS, Northeastern University
tai.ku@northeastern.edu

Tony Zhang
MSCS, Northeastern University
zhang.tony@northeastern.edu

Tsung-Yueh Lai
MSCS, Northeastern University
lai.ts@northeastern.edu

ABSTRACT

In this paper, we present a competitive two-player Tetris AI system built on a Deep Q-Network (DQN) framework. We implement a full Tetris engine supporting standard competitive mechanics, including combo chains, back-to-back bonuses, and a garbage queue with counter-attack cancellation. Our agent evaluates post-placement board features alongside piece type encodings and opponent state, using a placement-abstracted action space that enumerates all reachable positions for a given piece. We train three agent variants — offensive, defensive, and neutral — each driven by a different reward function, and evaluate them in a double round-robin tournament of 24 models. Results indicate that the design of the reward function directly shapes the agent's playstyle and competitive performance. The offensive agent achieved the highest overall win rate of 60% across the full tournament and 55.4% in the finale, while the defensive strategy proved the weakest, with win rates as low as 11%. Neutral agents performed competitively with final win rates between 48.0% and 49.0%, suggesting that balancing attack and defense is viable but does not fully match a dedicated offensive approach.

1. INTRODUCTION

Tetris is a classic puzzle game in which players place a continuous stream of falling tetrominoes to clear horizontal lines on a fixed grid. Players control seven tetromino types, each composed of four blocks, rotating and moving them horizontally before they land. Completed lines are cleared, and the player earns points. The game ends when the stack reaches the top of the grid. Although the rules are simple, the number of possible game states is enormous. A standard 10×20 board, combined with seven piece types, their rotations, and mechanics like holding pieces, creates a massive search space. Because of this complexity, Tetris has long been a popular testbed for artificial intelligence (AI) research [1].

Most previous work on Tetris AI focuses on the single-player version, where rule-based heuristics and search methods can already achieve strong performance. However, two-player competitive Tetris introduces a new layer of complexity. When a player clears multiple lines, they send "garbage" rows to their opponent, making the opponent's board harder to manage. The amount of garbage sent depends on how many lines are cleared at once: clearing two lines sends one row, clearing three lines sends two, and clearing four lines (Tetris) sends four. Consecutive clears (combos) send additional garbage on top of this. Crucially, incoming garbage can be canceled by attacking before it lands, which means the timing and aggression of a player's moves directly affect how much pressure they can absorb or deflect. As a result, an effective agent must do more than just survive; it must also apply pressure. Balancing these two objectives is challenging, which is why reinforcement learning is a natural fit, as agents can learn this balance directly from experience.

Prior work on single-player Tetris AI has explored heuristic-based placement search [2], [3] and neural network approaches, achieving strong performance in survival-based settings. In this paper, we develop a full two-player Tetris engine based on mechanics used in modern Tetris games, including SRS wall kicks, DAS/ARR input handling, a hold system, combo chains, back-to-back bonuses, and a garbage queue with counter-attack cancellation. On top of this environment, we train a Deep Q-Network (DQN) agent [4] based on the architecture proposed by van der Ree and Wiering [5], adapted for competitive play. Instead of using raw board states, the agent evaluates post-placement features such as aggregate height, number of holes, and surface bumpiness, along with one-hot encodings of the current and held pieces and the opponent's board height.

We train three variants of the agent: offensive, defensive, and neutral. Each uses a different reward function. These agents are then evaluated in a double round-robin tournament. Our results show that the offensive agent, which learns to aggressively build and chain combos, achieves the highest overall win rate.

2. METHODS

a. Deep Q-Network (DQN)

In this section, we introduce each component of our DQN framework. Figure 1 provides an overview of the full system pipeline.

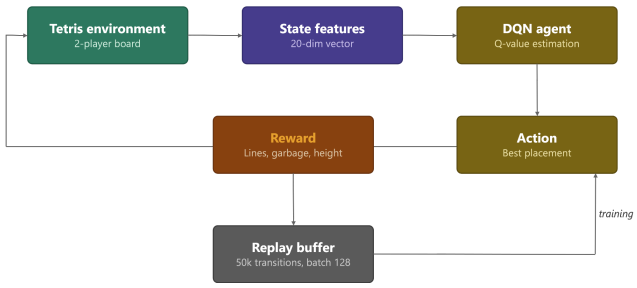

Figure 1: System pipeline overview of the DQN-based competitive Tetris agent.

i. State-Action Space

Instead of feeding state-action pairs separately, we enumerate all possible actions, simulate each action, score each resulting state with the network, and pick the best action to perform. To generate all possible placements for a given piece, a Pathfinder module iterates over all valid rotations and column positions, simulates a hard drop at each location, and records the resulting board state along with the command sequence needed to reach it. This placement abstraction reduces the action space to only the reachable final positions, significantly simplifying what the network needs to learn.

Each state is represented as a 20-dimensional vector consisting of four board features, a 7-dimensional one-hot encoding of the current piece type, a 7-dimensional one-hot encoding of the held piece type, the opponent's maximum board height, and a binary flag indicating whether hold is available. The four board features are aggregate maximum height, holes, bumpiness, and lines cleared by the action [2]. A hole is defined as a space such that there is at least one tile in the same column above it. A hole is harder to clear because the agent must first remove all the lines above it before it can reach and fill it, so minimizing holes is essential. The bumpiness of a grid indicates the variation in its column heights. It is computed by summing up the absolute differences between all adjacent columns [3]. For each encountered piece, the agent can choose to place or hold it. If there is already a held piece, the current piece will be swapped with the held one, and holding will be prohibited for one piece. Therefore, the board layout, the availability of holding, the held piece, and the current piece all work together to define the action space.

Each action consists of a series of game commands. To reduce the computational complexity and the Pathfinder's responsibility, we exclude the agent's ability to make T-spins, a move that modern Tetris games reward players for twisting the T tetromino into tight spaces. Therefore, the series of commands should only contain rotations, left and right movements, hold, and end with an instant drop to place the piece.

ii. Q-Learning

Like a normal DQN framework, the update is based on the Bellman Equation [4]:

$$Q(s,\ a) \leftarrow Q(s,\ a)\ +\ \alpha(r\ +\ \gamma\, max_{a'} Q(s',\ a';\ \theta^{-})\ -\ Q(s,\ a;\ \theta))$$

where θ are the weights of the main Q-network and θ^{-} are the weights of the target Q-network. To stabilize training, we maintain a separate target network whose weights are only updated every 200 training steps, preventing the agent from chasing a constantly shifting target. Transitions are stored in a replay buffer of size 50,000, and at each training step, a random mini-batch of 128 samples is drawn to perform a gradient update. Training does not begin until 1,000 transitions have been collected. The network itself consists of three fully connected layers of size 64, 64, and 32, followed by a single linear output neuron representing the Q-value of that state. We use Huber loss and the Adam optimizer with a learning rate of 0.001. Each agent is trained for 2,500 episodes with epsilon decaying linearly from 1.0 to 0.05 over the first 2,000 episodes, after which it remains fixed. Figure 2 illustrates the network architecture.

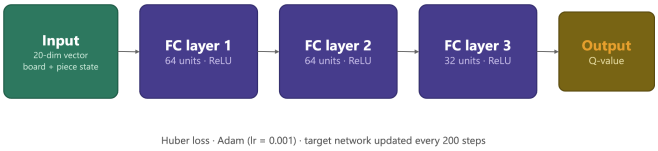

Figure 2: Network architecture of the DQN agent.

iii. Rewards Function

The rewards function varies depending on the agent's playing strategies. To experiment with the effectiveness of different strategies, two reward configurations with different sensitivities to changes in board height, the number of lines cleared, the number of garbage lines sent, and holes and bumpiness were used to train each agent style. Overall, three strategies were deployed: (i) offensive, (ii) defensive, and (iii) neutral. The offensive agent prioritizes sending garbage lines to the opponent and tries to kill the opponent by stacking its height. The defensive agent focuses on reducing its own height. It disregards the advantage of sending garbage lines by clearing; instead, the main goal is to keep its board as low as possible. The neutral agent is a combination of both offensive and defensive. It strikes a balance between aggressive attack and conservative defense.

$$r \leftarrow LR(l)\ +\ GS(l)\ -\ w_1 holes\ -\ w_2 bumpiness\ -\ w_3^{\pm}\delta(h)^{\pm}$$

Line reward (LR) maps the number of lines cleared to rewards, garbage sent (GS) maps the number of garbage lines sent to rewards, and $\delta(h)^{\pm}$ is the height change. Rather than penalizing absolute board height, we use a delta-based height penalty that distinguishes between height increases and decreases: increasing height is penalized more heavily than decreasing height is rewarded. This prevents the agent from learning to die quickly to avoid height penalties. Adjusting LR, GS, and weights w_i results in different sensitivities and agent strategies.

b. Training Opponents

The opponent used to play against the training agent is pivotal. It significantly affects the agent's performance. Four opponent strategies were used: (i) random, (ii) agent (self-play), (iii) heuristic, and (iv) hybrid. To further improve training coverage, each episode also begins with the agent's board pre-filled with a random number of garbage lines between 0 and 14, exposing the agent to high board states and late-game scenarios that would otherwise be rarely encountered.

i. Random

The opponent always chooses a random move from the current action space. This mimics an agent performing constant exploration without epsilon decay.

ii. Agent (Self-play)

The opponent is the agent itself, using the target network to make action decisions to avoid chasing a constantly changing goal.

iii. Heuristic

The heuristic opponent uses a heuristic function to compute heuristic values based on the current state. The action of the largest heuristic value will be performed.

$$h_a \leftarrow HLR(L) - P_a$$

$$P_a \leftarrow 3 \times holes + 0.2 \times bump + 0.5 \times max_h + 0.1 \times agg_h$$

where heuristic line reward (HLR) maps the number of lines cleared to heuristic values, and P_a denotes the penalty caused by acting a. Penalty consists of holes, bumpiness (bump), maximum height (max_h), and aggregate board height (agg_h).

iv. Hybrid

The hybrid opponent is the combination of a heuristic and an agent. When the training agent focuses on exploration, the heuristic opponent is deployed to provide a more stable playstyle and a clear goal for the agent to adapt to. After the stop episode, where epsilon stops decaying, the opponent is replaced by the target network.

c. Model Tournament

To compare the performance and competitiveness of 24 models, we run a double-round-robin tournament with 100 games per matchup to simulate gameplay. To ensure the agents will not play endlessly before either of them dies, a maximum piece usage limit is applied to prevent excessive game duration. Once the maximum usage is reached, the game result is determined by each player's score. Moreover, weaker models tend to allow others to accumulate inflated statistics, which means the results of the full tournament may not accurately reflect the agents' true competitive performance. As a result, we initiated a second tournament with the top 5 models. By eliminating weaker candidates, we achieve a more authentic comparison and reduce the redundancy of model candidates.

The collected states are used to analyze the strength, weakness, and playstyle of each model: (i) Win Rate, (ii) Average Score per Game, (iii) Average Line Cleared per Game, (iv) Average Garbage Sent per Game, (v) Average Line Cleared per Piece, (vi) Singles / Doubles / Triples / Tetrises, (vii) Average Combo per Game.

3. RESULTS

This section presents the training and evaluation results of the 24 trained models. We first analyze the DQN reward curves during training, followed by the results of the double round-robin tournament and the tournament finale.

a. DQN Rewards

All eight offensive agent configurations showed a consistent upward trend in cumulative rewards throughout the training process. Models trained against random opponents (Figures 3, 4) peaked around 20,000, while models trained against agent opponents (Figures 5, 6) and heuristic opponents (Figures 7, 8) reached notably higher reward peaks, exceeding 50,000 and 60,000, respectively. This suggests that more challenging training opponents push the offensive agent to develop stronger clearing and garbage-sending behaviors. Models trained against hybrid opponents (Figures 9, 10) exhibited steady reward growth, benefiting from the transition between heuristic and self-play opponents during training.

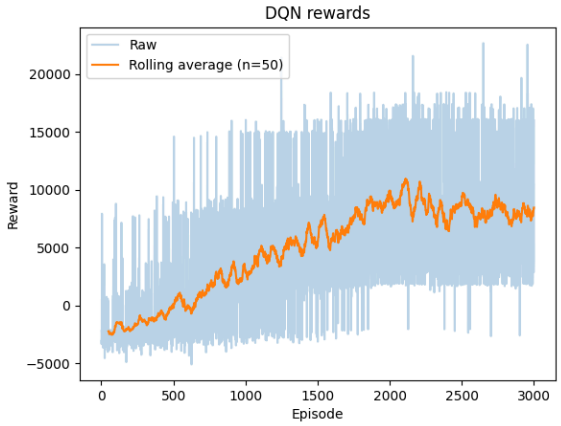

Figure 3: Offensive Agent v.s. Random Opponent v1 (OFRA1).

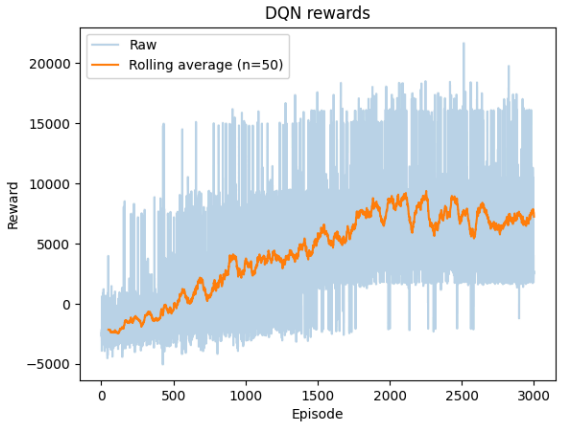

Figure 4: Offensive Agent v.s. Random Opponent v2 (OFRA2).

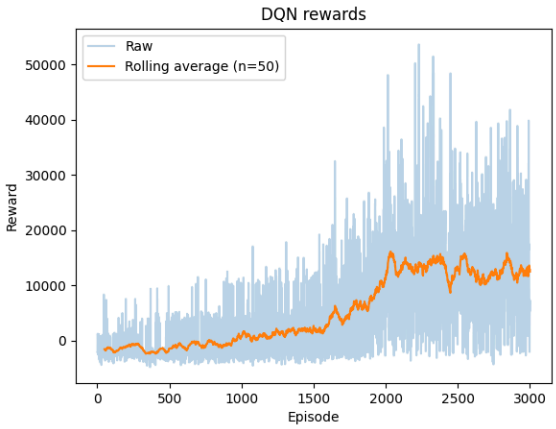

Figure 5: Offensive Agent v.s. Agent Opponent v1 (OFAG1).

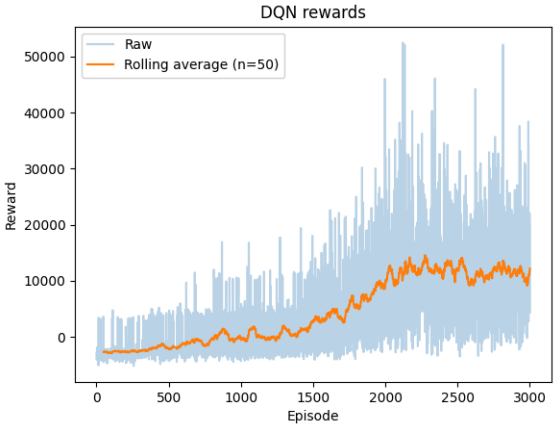

Figure 6: Offensive Agent v.s. Agent Opponent v2 (OFAG2).

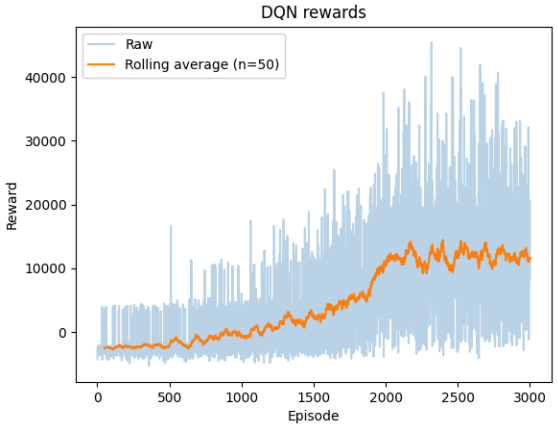

Figure 9: Offensive Agent v.s. Hybrid Opponent v1 (OFHY1).

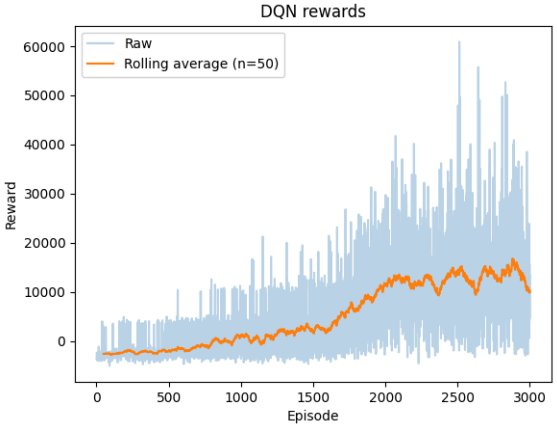

Figure 7: Offensive Agent v.s. Heuristic Opponent v1 (OFHE1).

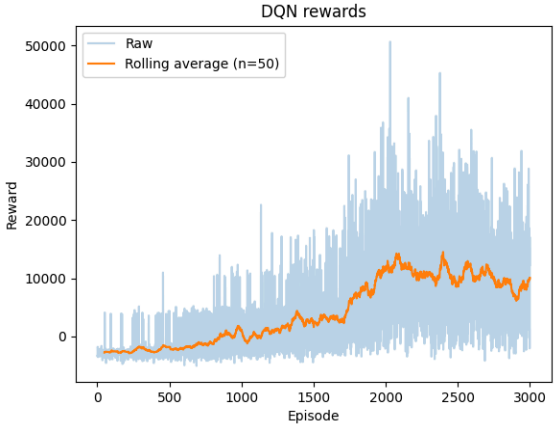

Figure 10: Offensive Agent v.s. Hybrid Opponent v2 (OFHY2).

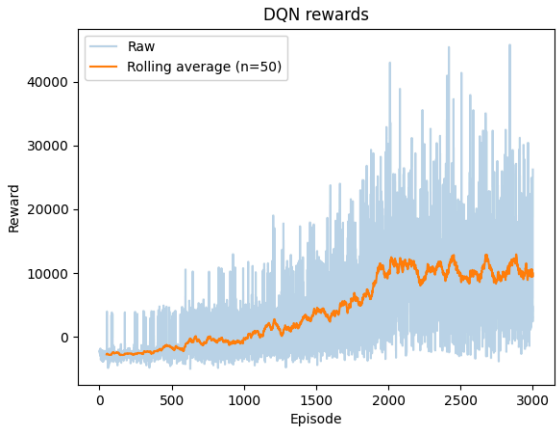

Figure 8: Offensive Agent v.s. Heuristic Opponent v2 (OFHE2).

The eight defensive agent configurations showed a notably different training pattern compared to the offensive agents. Models trained against random opponents (Figures 11, 12) exhibited relatively low reward peaks at around 12,500 and 4,000, respectively, with DFRA2 showing slower convergence. Models trained against agent opponents (Figures 13, 14) reached moderate reward peaks around 25,000 and 15,000, while models trained against heuristic opponents (Figures 15, 16) showed the most inconsistent training behavior, with DFHE1 peaking around 25,000 but DFHE2 struggling to exceed 10,000. Models trained against hybrid opponents (Figures 17, 18) showed a more stable upward trend, though reward peaks remained lower than those of their offensive counterparts, reflecting the defensive strategy's inherent limitation of not prioritizing aggressive line clearing.

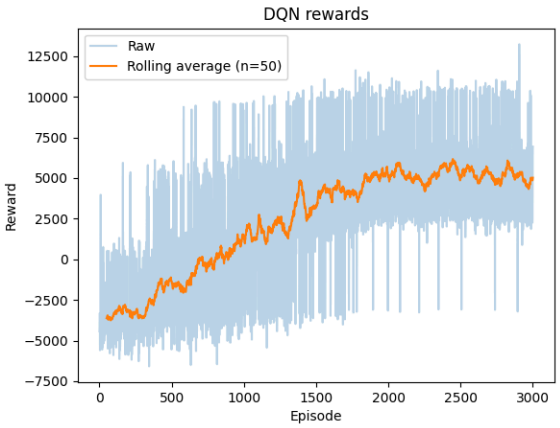

Figure 11: Defensive Agent v.s. Random Opponent v1 (DFRA1).

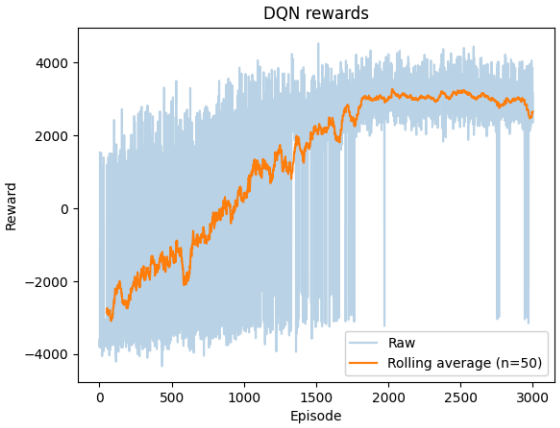

Figure 12: Defensive Agent v.s. Random Opponent v2 (DFRA2).

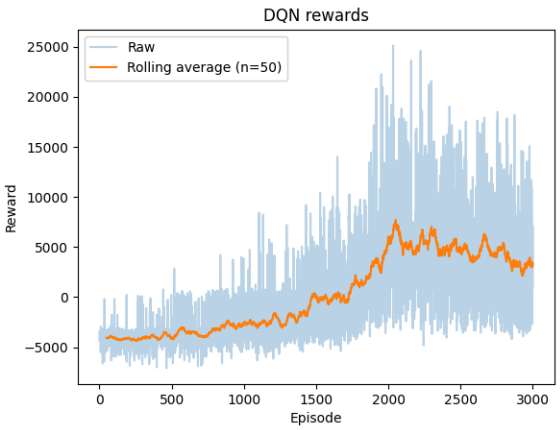

Figure 13: Defensive Agent v.s. Agent Opponent v1 (DFAG1).

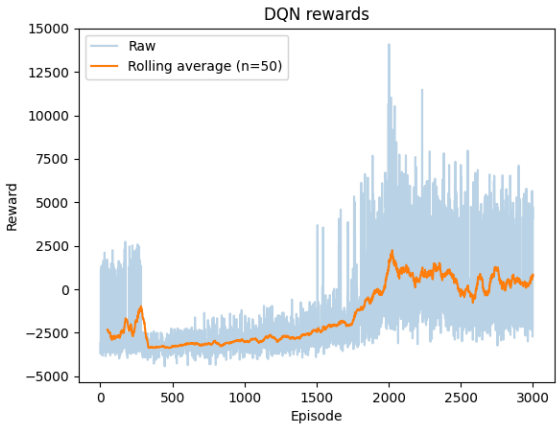

Figure 14: Defensive Agent v.s. Agent Opponent v2 (DFAG2).

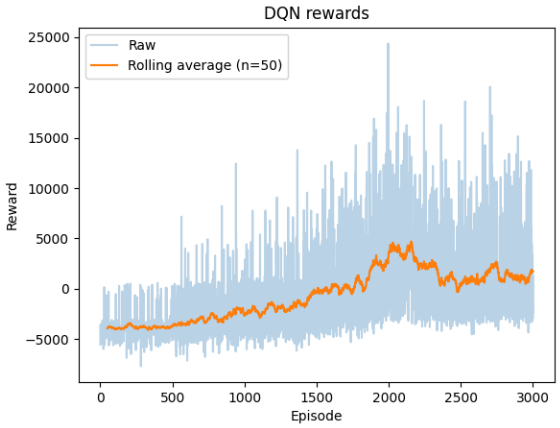

Figure 15: Defensive Agent v.s. Heuristic Opponent v1 (DFHE1).

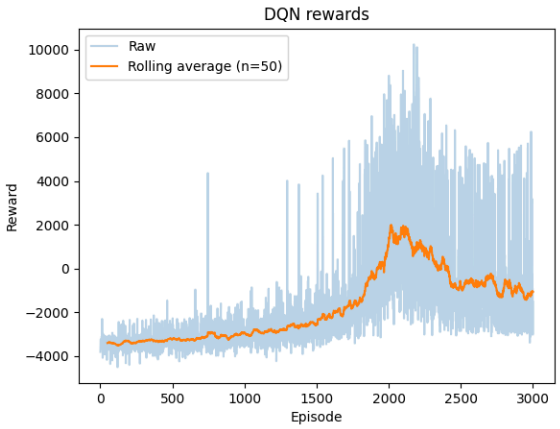

Figure 16: Defensive Agent v.s. Heuristic Opponent v2 (DFHE2).

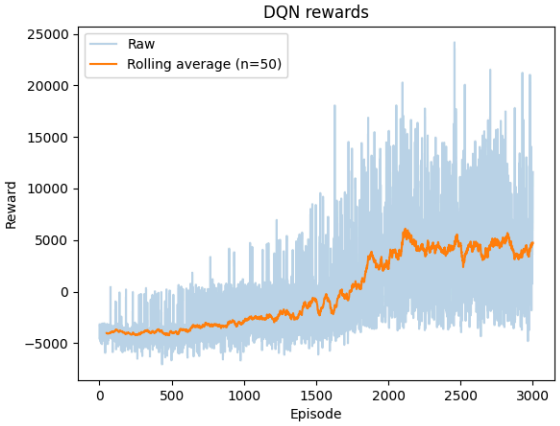

Figure 17: Defensive Agent v.s. Hybrid Opponent v1 (DFHY1).

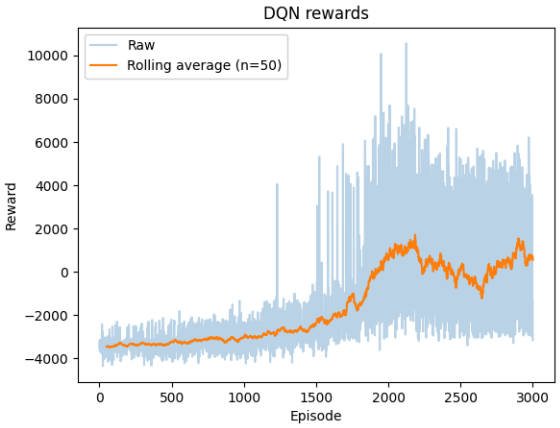

Figure 18: Defensive Agent v.s. Hybrid Opponent v2 (DFHY2).

The eight neutral agent configurations exhibited training behavior that fell between that of the offensive and defensive agents. Models trained against random opponents (Figures 19, 20) showed steady reward growth, peaking around 15,000 and 12,500, respectively. Models trained against agent opponents (Figures 21, 22) reached higher reward peaks around 40,000 and 35,000, while models trained against heuristic opponents (Figures 23, 24) showed the strongest performance among neutral configurations, with NEHE1 peaking around 40,000. Models trained against hybrid opponents (Figures 25, 26) also showed consistent upward trends, peaking around 40,000 and 30,000, respectively. Overall, neutral agents benefited from more challenging opponents, as did offensive agents, suggesting that the balanced reward structure learned effective strategies from diverse training environments.

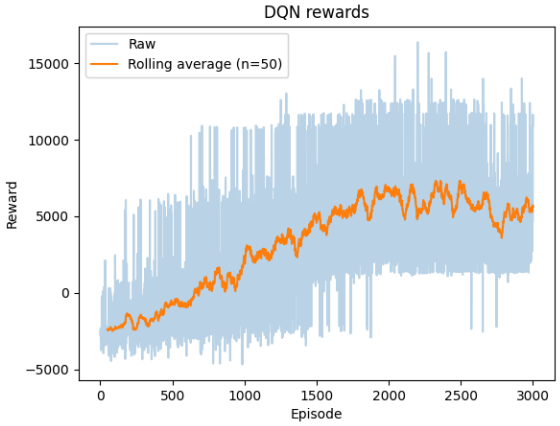

Figure 19: Neutral Agent v.s. Random Opponent v1 (NERA1).

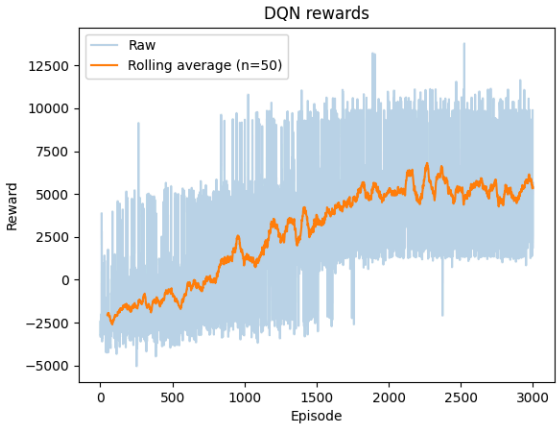

Figure 20: Neutral Agent v.s. Random Opponent v2 (NERA2).

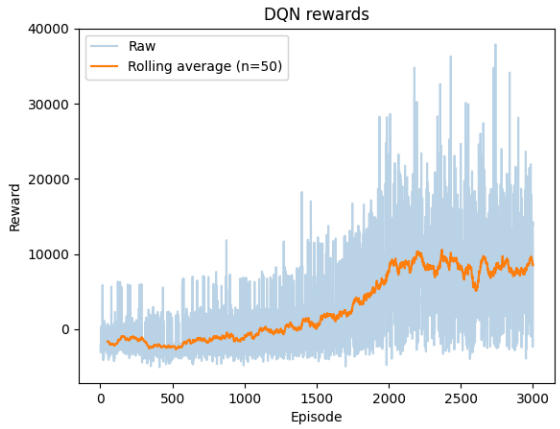

Figure 21: Neutral Agent v.s. Agent Opponent v1 (NEAG1).

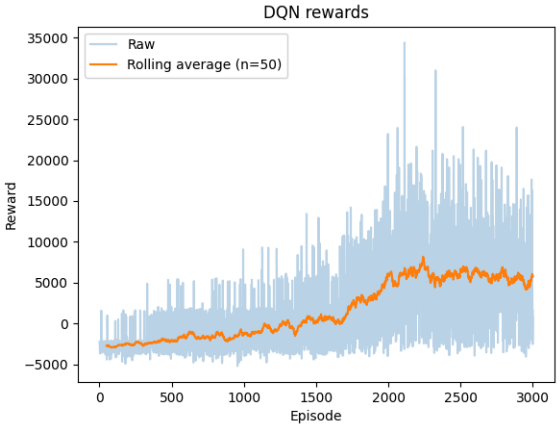

Figure 22: Neutral Agent v.s. Agent Opponent v2 (NEAG2).

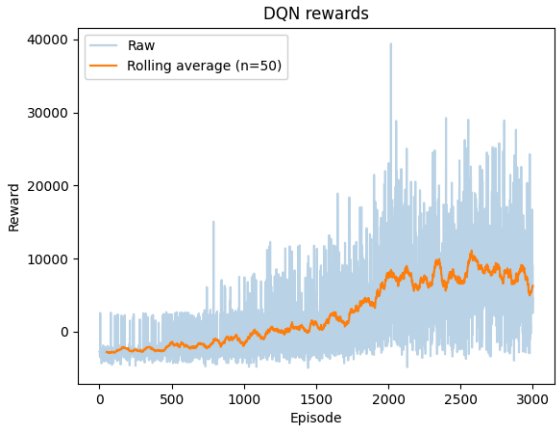

Figure 23: Neutral Agent v.s. Heuristic Opponent v1 (NEHE1).

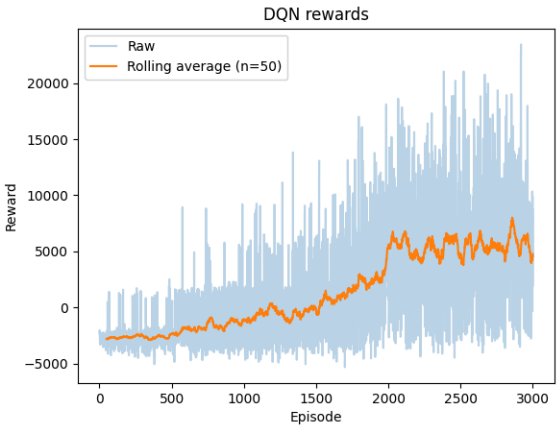

Figure 24: Neutral Agent v.s. Heuristic Opponent v2 (NEHE2).

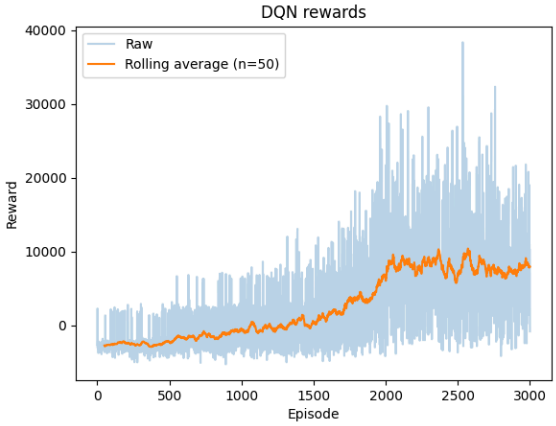

Figure 25: Neutral Agent v.s. Hybrid Opponent v1 (NEHY1).

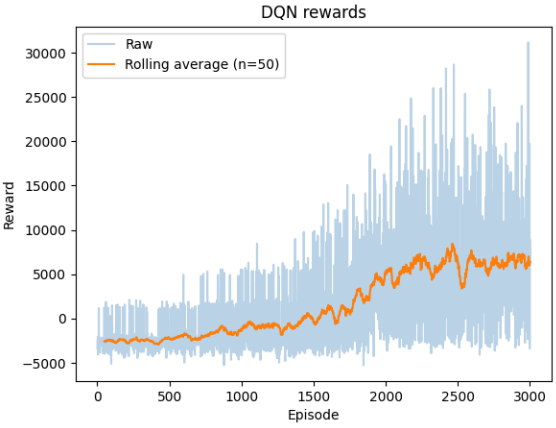

Figure 26: Neutral Agent v.s. Hybrid Opponent v2 (NEHY2).

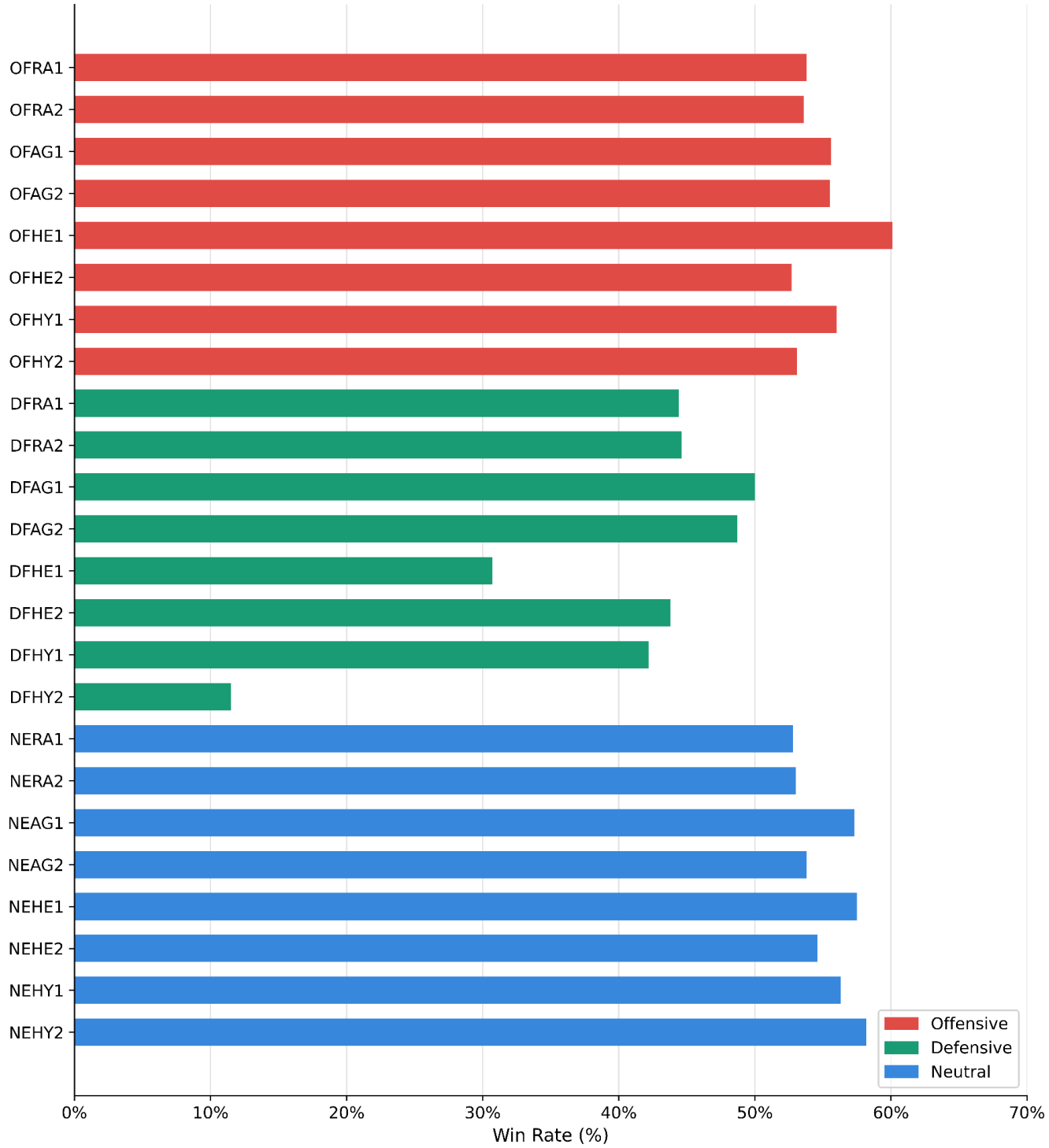

Figure 27. The win rates of 24 models in the double round robin tournament.

Table I: Summary of Model Performance during Tournament Finale

Finale	Win Rate (%)	Avg. Score	Avg. Lines	Avg. Garbage	Lines Per Piece
OFHE1	55.4	10825.3	70.2	41.4	0.544
NEHY1	49.0	10262.8	65.9	38.6	0.532
NEAG1	48.9	10438.4	67	39.5	0.534
NEHY2	48.8	10284.6	66	38.8	0.534
NEHE1	48.0	10141.1	64.6	38.3	0.531

Table II: Summary of Model Clearing Stats Average during Tournament Finale

Finale	Singles	Doubles	Triples	Tetrises	Avg. Combos
OFHE1	45.95	13.53	1.46	1.55	22.6
NEHY1	45.60	12.83	1.08	1.27	21.9
NEAG1	44.07	13.05	1.41	1.42	21.4
NEHY2	44.95	12.86	1.18	1.31	21.9
NEHE1	43.95	12.68	1.21	1.24	21.4

b. Tournament Analysis

i. First Round Tournament

Figure 27 shows the win rates of all 24 models across the double round-robin tournament, with each model playing 4,600 games in total. Overall, defensive models are the worst, consistently falling in the 30–50% win-rate range, with DFHY2 recording the lowest win rate of 11%. Offensive models clustered between 52% and 60%, with OFHE1 achieving the highest win rate of 60%. Neutral models performed competitively, ranging between 52–58%. Based on these results, we selected the top five models — OFHE1, NEHY1, NEHY2, NEHE1, and NEAG1 — to participate in the tournament finale and to collect more detailed statistics.

ii. Tournament Finale

Tables I and II summarize the performance and behavior statistics of the top five models in the finale. Table I shows that OFHE1 dominated, with a final win rate of 55.4%, an average score of 10,825.3, and the highest average number of garbage lines sent at 41.4, confirming its aggressive playstyle. The neutral models followed closely, with win rates ranging from 48.0% to 49.0% and average scores between 10,141.1 and 10,438.4. Table II reveals that OFHE1 also led in tetrises with 1.55 and combos with 22.6 per game, while the neutral models recorded slightly lower values across all clearing categories. The relatively small gap between triples and tetrises across all models suggests that agents learned to prioritize high-value clears, as tetrises send double the garbage lines compared to triples, making them a significantly more powerful attacking tool.

4. CONCLUSIONS

In this work, we investigated how different reward function designs influence agent behavior and competitive performance in two-player Tetris. Our experiments across 24 trained models show that the choice of reward function significantly impacts both playstyle and win rate.

The offensive strategy proved to be the most effective, with OFHE1 achieving the highest finale win rate of 55.4% and the highest average garbage lines sent per piece at 0.544. This confirms that prioritizing aggressive multi-line clearing, combined with combo chaining, is the dominant strategy in competitive Tetris, as it simultaneously reduces the agent's board height while pressuring the opponent with garbage lines.

The defensive strategy was consistently the weakest, with win rates as low as 11%. Focusing solely on reducing board height without attacking creates a losing feedback loop — the agent survives longer but never applies enough pressure to defeat opponents who aggressively clear lines.

The neutral agents demonstrated that balancing attack and defense is a viable approach, finishing with final win rates of 48.0%-49.0%. However, the consistent gap between garbage sent and tetris count, compared to OFHE1, suggests that in competitive Tetris, a pure offensive strategy edges out a balanced one.

Future work could explore T-spin support to unlock more advanced attacking moves, more sophisticated opponent modeling to allow dynamic adaptation during matches, and curriculum learning to expose agents to a wider range of opponent skill levels during training.

5. REFERENCES

[1] E. D. Demaine, S. Hohenberger, and D. Liben-Nowell, "Tetris is hard, even to approximate," in *Proc. 9th International Computing and Combinatorics Conference (COCOON)*, 2003, pp. 351–363.

[2] N. Faria, "Tetris AI," 2018. [Online]. Available: https://github.com/nuno-faria/tetris-ai.

[3] Y. Lee, "Tetris AI – The (Near) Perfect Bot," Code My Road, Apr. 14, 2013. [Online]. Available: https://codemyroad.wordpress.com/2013/04/14/tetris-ai-the-near-perfect-player/.

[4] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015. doi: 10.1038/nature14236.

[5] M. van der Ree and M. Wiering, "Reinforcement learning in the game of Othello: Learning against a fixed opponent and learning from self-play," in *Proceedings of the 2013 IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning (ADPRL)*, Singapore, 2013, pp. 108–115. doi: 10.1109/ADPRL.2013.6614996.